Poncho Digital

Documentación del proyecto y funcionalidades finales · Laboratorio de Programación III · Ingeniería en Informática · U.N.Ca. · Noviembre de 2026

1. Descripción

Poncho Digital es una plataforma web que brinda información sobre los artesanos y productores de la Fiesta Nacional e Internacional del Poncho: permite buscar artesanos y productos y ubicar los stands. Está desarrollada con **Django 6.1** y **PostgreSQL**, con la arquitectura MTV (modelo, template, vista) de Django.

Aplicación	Responsabilidad
`usuarios`	Usuario propio (`AbstractUser`) con rol; grupos y permisos (`roles.py`)
`artesanos`	Artesanos, rubros, localidades y solicitudes de postulación
`productos`	Catálogo: listado, alta, edición, baja y reportes
`stands`	Stands asignados a artesanos aprobados

2. Puesta en marcha

```
py -3.12 -m venv env_poncho
.\env_poncho\Scripts\Activate.ps1
pip install -r requirements.txt

# crear la base "poncho_digital" en PostgreSQL y el archivo .env
# (copiar .env.example y completarlo)

python manage.py migrate
python manage.py cargar_datos_ejemplo
python manage.py runserver          # http://127.0.0.1:8000/
```

- El archivo `.env` contiene las credenciales de la base y **no se sube a Git** (está en `.gitignore`); el repositorio incluye `.env.example` como modelo.
- `cargar_datos_ejemplo` puede ejecutarse varias veces: no duplica datos.

3. Roles y permisos

Rol	Usuario de demo	Qué puede hacer
Administrador	`admin`	Evaluar solicitudes, asignar y eliminar stands, gestionar cualquier producto, reportes, admin de Django
Artesano	`30123456`, `28456789`	Alta, modificación y baja de **sus** productos; ver stands
Postulante	`35678901`, `32109876`	Ver el estado de su solicitud
Visitante	`visitante1`	Ver el catálogo (también sin iniciar sesión)

- Las contraseñas de demo figuran en el `README.md` del repositorio.
- El campo `rol` es una etiqueta; los **permisos reales los dan los grupos** de Django (`Visitante`, `Artesano`, `Administrador`), definidos en `usuarios/roles.py`. Al cambiar el rol de un usuario se lo pasa automáticamente al grupo correspondiente.

4. Funcionalidades incorporadas en la etapa final

4.1 Reporte parametrizado de productos (solo administradores). Ruta: `/productos/reporte/`.

Parámetro (GET)	Efecto
`q`	Texto contenido en el nombre del producto
`rubro`	Filtra por rubro
`artesano`	Filtra por artesano
`precio_min`, `precio_max`	Rango de precio
`orden`	Nombre, precio (ascendente o descendente) o artesano
`descargar=1`	En lugar de la vista previa, descarga el resultado en CSV

- El CSV usa `;` como separador y BOM UTF-8, de modo que **Excel abre las tildes correctamente**. Columnas: Producto, Rubro, Precio, Artesano, Localidad.
- La pantalla muestra una vista previa con el mismo filtro que luego se exporta, y un enlace "Exportar reporte" desde el listado de productos (solo visible para administradores).

4.2 Reporte por artesano. Ruta: `/productos/reporte/artesano/<id>/`. Descarga el CSV con los productos de un solo artesano (`reporte_<nombre>.csv`). El reporte general lista a los artesanos aprobados con su cantidad de productos y un enlace de descarga por cada uno.

4.3 Eliminación de un stand asignado. Ruta: `/stands/<id>/eliminar/`. Botón en el listado de stands, con confirmación en el navegador. Se ejecuta **solo por POST** y exige el permiso `stands.delete_stand`.

4.4 Imágenes de productos. Comando `python manage.py generar_imagenes_productos` (con `--todos` para reemplazar también las existentes). Genera con Pillow una **ilustración** para cada producto sin imagen. No son fotografías: las fotos reales se cargan desde el formulario de modificación del producto.

5. Seguridad implementada

Medida	Implementación
Autenticación	`@login_required` en todas las vistas de gestión
Autorización	`@permission_required(..., raise_exception=True)`: sin permiso se responde 403
Propiedad	Un artesano solo modifica o elimina sus propios productos
Rol de administrador	Los reportes verifican además `es_administrador(request.user)`
Acciones destructivas	`@require_POST` en la baja de stands (405 si se pide por GET)
CSRF	`{% csrf_token %}` en todos los formularios `POST`
XSS	Autoescape de los templates de Django
Inyección SQL	Consultas por medio del ORM (parametrizadas)
Secretos	Credenciales en `.env`, fuera del repositorio
Interfaz	Los botones se ocultan con `{% if perms... %}`, pero la protección real está en la vista

6. Mapa de rutas principales

Ruta	Descripción
`/`	Inicio
`/ingresar/`, `/salir/`	Inicio y cierre de sesión
`/artesanos/...`	Postulación, solicitudes y listado de artesanos
`/productos/`, `agregar/`, `<id>/editar/`, `<id>/eliminar/`	Catálogo y gestión de productos
`/productos/reporte/`, `reporte/artesano/<id>/`	Reportes (administrador)
`/stands/`, `agregar/`, `<id>/eliminar/`	Gestión de stands
`/admin/`	Administración de Django (usuarios, roles y grupos)

7. Limitaciones conocidas y mejoras futuras

- `DEBUG=True` y `ALLOWED_HOSTS=[]`: son valores de desarrollo; para publicar hay que configurarlos para producción.
- Las pruebas automáticas (`tests.py`) están vacías; sería conveniente cubrir permisos y reportes.
- Las imágenes de los productos de demo son ilustraciones generadas, no fotografías.
- Los reportes se exportan solo en CSV; se podrían agregar PDF o Excel.